

Hebrew Recipe RAG Report

Corpus Description

This project implements a custom Retrieval-Augmented Generation pipeline over a private Hebrew recipe corpus. The corpus contains local Hebrew culinary documents in PDF and DOCX format. After document-level deduplication, the working corpus contains **610** processed documents, which produce **875** chunks and **873** indexed chunks after filtering very short text.

The domain is suitable for RAG because recipe questions depend on exact retrieval of ingredients, quantities, cooking times, temperatures, and preparation steps. The corpus also contains duplicated and near-duplicated recipes, source-specific recipe titles, and general guide documents, all of which create realistic retrieval challenges.

System Architecture

The pipeline is implemented manually in Python without LangChain or LlamaIndex wrappers. The main stages are:

1. scan the raw corpus
2. load PDF and DOCX documents
3. clean and normalize Hebrew text
4. deduplicate documents
5. create chunks
6. build dense and lexical indexes
7. retrieve relevant chunks
8. generate a grounded answer with citations
9. evaluate retrieval and manually inspect answer quality

The required public interface is:

```
def answer(question: str) -> dict:
    return {
        "answer": str,
        "sources": list[str],
        "retrieved_chunks": list[dict]
    }
```

Chunking Strategy

Two chunking strategies were considered during development:

- full-document chunking
- fixed-size chunking with overlap

The working system uses fixed-size chunking with:

- chunk size: 300 words
- overlap: 50 words

Each chunk preserves source metadata, and chunk IDs remain stable. This strategy performed better than relying on whole documents because recipe answers are often localized to specific sections such as ingredients, proofing instructions, or baking temperatures. Overlap helps preserve short multi-step instructions that otherwise fall across chunk boundaries.

Embedding and Vector Index Choice

The dense retrieval model is `intfloat/multilingual-e5-small`, which is appropriate for multilingual text including Hebrew. E5 query and passage prefixes are used consistently:

- `query: <question>`
- `passage: <chunk text>`

Dense vectors are normalized and indexed in FAISS using `IndexFlatIP`. The final system also introduces `indexed_text`, which combines:

- source filename
- category
- original chunk text

This metadata-aware representation improved retrieval for recipe names that appear in filenames or categories but not in the chunk body.

Retrieval Method

The project moved through four main retrieval stages:

Experiment	Retrieval setup	Hit@5	Hit@20	MRR
Dense baseline after gold/dedup fixes	Dense FAISS	68%	82%	0.4857
BM25	BM25 over text	78%	88%	0.7155
Weighted hybrid before metadata	Dense + BM25 + weighted RRF	82%	88%	0.7248
Metadata-aware weighted hybrid	<code>indexed_text</code> + hybrid	92%	94%	0.8412

The final recommended configuration is weighted hybrid retrieval:

```
python eval/run_eval.py --mode hybrid --candidate-k 50 --rrf-k 30 --dense-weight 0.5 --bm25-weight 2.0
```

This combines:

- dense FAISS retrieval
- BM25 lexical retrieval
- Reciprocal Rank Fusion

BM25 was especially useful for exact recipe names, ingredients, quantities, and cooking times. Dense retrieval alone often found relevant chunks by top 10 but ranked them too low.

Prompt Design and Answer Generation

Answer generation uses a local Ollama model with a grounded Hebrew-only prompt. The generation stage was improved substantially over the course of the project. The final behavior includes:

- answer only from retrieved context
- Hebrew-only response
- chunk or source citation
- strict fallback when the answer is unsupported:
 - `לא מצאתי את המידע במקורות שנשלפו.`
- source-neighbor context expansion
- recipe-specific context filtering

The context filtering step became important after diagnostics showed that the model could answer from irrelevant but semantically related recipes such as hamburger or bolognese when the user explicitly asked about `אינגריי`. The final generation logic prioritizes chunks from matching recipe sources and keeps unrelated sources from dominating the prompt when a clear recipe name is present.

Evaluation Methodology

Retrieval evaluation uses `eval/gold_set.jsonl` with `50` manually curated Hebrew questions. Each gold row contains:

- `question`
- `reference_answer`
- `must_cite_chunk_ids`
- `category`

The evaluator reports:

- Hit@1
- Hit@3
- Hit@5
- Hit@10
- Hit@20
- MRR
- average first relevant rank

It also reports source-aware and neighbor-aware diagnostic Hit@K values, which help separate true retrieval failures from chunk-boundary issues.

Final retrieval results for the best system:

- Hybrid Hit@1: 78%
- Hybrid Hit@3: 90%
- Hybrid Hit@5: 92%
- Hybrid Hit@10: 94%
- Hybrid Hit@20: 94%
- MRR: 0.8412
- misses_at_20 : 3

Ablation Study

The most useful ablation path was retrieval-oriented rather than chunk-size-only tuning:

Stage	Change	Hit@5	Hit@20	MRR	Notes
Dense baseline after gold/dedup fixes	Dense FAISS	68%	82%	0.4857	ranking and recall limitations
BM25	BM25 over text	78%	88%	0.7155	strong lexical matching
Weighted hybrid before metadata	Dense + BM25 + weighted RRF	82%	88%	0.7248	best pre-metadata setup
Metadata-aware weighted hybrid	indexed_text + hybrid	92%	94%	0.8412	current best

Main conclusions from the ablation path:

- dense retrieval alone was not enough
- BM25 materially improved lexical questions
- hybrid retrieval improved over dense and became strongest after weighting and metadata-aware indexing
- metadata made a large difference for recipe-title recall

Failure Analysis

Early failure analysis found several recurring issues:

1. relevant chunks were often retrieved below top 5
2. some misses were adjacent-chunk issues rather than full retrieval failures
3. recipe names such as אינגריי existed in metadata but not in chunk text
4. bread recipe chunks were confused with general bread and gluten guides
5. answer generation could use the wrong retrieved context even when the right chunk was present

The combination of analyze_misses.py , source and neighbor metrics, manual answer evaluation, and query diagnostics was important for separating retrieval bugs from generation bugs.

Manual answer evaluation on 10 selected questions showed major improvement:

Before generation and context fixes:

- correct: 0

- partial: 3
- incorrect: 7

After generation and context fixes:

- correct: 8
- partial: 2
- incorrect: 0
- unsupported / hallucinated: 0

The remaining partial cases are Q6 and Q7, where the system still falls back on gluten-free bread proofing questions instead of extracting the answer confidently.

Future Improvements

The current system is a strong mid-course baseline, but several improvements remain useful:

- stronger recipe-title matching for bread and gluten-related questions
- section-aware chunking for ingredient lists vs preparation steps
- reranking over hybrid retrieval candidates
- larger manual answer evaluation set
- broader ablations over chunk size and overlap
- optional answer-specific post-processing for numerical and temporal recipe questions

Overall, the project demonstrates that retrieval quality and answer quality must be evaluated separately. The final system performs well because it combines metadata-aware retrieval, lexical+dense fusion, and stricter grounded generation behavior.